

Operációs rendszerek II.

UNIX: fájlrendszerek



Várkonyiné Kóczy Annamária

Professzor

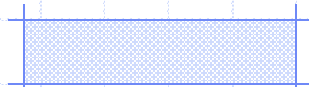
Informatika Tanszék

koczya@uj.ssk

varkonyi-koczy@uni-obuda.hu

Felhasznált irodalom:

- Kóczy-Kondorosi (szerk.): Operációs rendszerek mérnöki megközelítésben
- Tanenbaum: Modern Operating Systems 2nd. Ed.
- Silberschatz, Galvin, Gagne: Operating System Concepts



16. Fájrendszer

- Ismétlés, bevezetés
- Alapfogalmak
- UNIX fájrendszer történeti áttekintése
- A fájlrendszer felhasználói szemmel
- Fájrendszer szervezése
- A System V fájlrendszer
- Berkeley FFS
- Swapping
- Virtuális memóriakezelés
- Copy-on-Write technika
- Hálózati és elosztott szolgáltatások a UNIX-ban
- virtuális fájlrendszer felépítés, alkalmazási példa: NFS

Bevezetés - ismétlés

- A folyamatok
 - a felhasználói programok futás alatt álló példányai
 - a programokat permanens tárból töltjük be
- A permanens táruk
 - nem felejtő, nagyságrendekkel nagyobb és lassabb a memóriánál
 - blokkos fizikai tárolás és fájl-alapú logikai szervezés
 - többféle megoldás egyedi jellemzőkkel (HDD, flash, usb, RAID, ...)
- A kernel
 - kezeli a hardver erőforrásokat (köztük a permanens tárukat)
 - a hardverkezelő réteg felett többszintű fájlrendszer réteg található
 - háttértár kezelés, fájlrendszer szervezés, logikai felépítés (fájl, könyvtár)
 - adminisztrálja a fájlok blokkjait és az üres helyeket a permanens tárban
 - elvégzi a fizikai és logikai szervezés közötti leképezést
 - programozói interfészt nyújt az alkalmazásfejlesztők számára

Alapfogalmak

- Fájl (file, állomány)
 - adattárolási hely
- Fájlrendszer (állományrendszer)
 - fájlok tárolásának szervezése, hozzáférés biztosítása
- Fájlrendszerek felhasználói felülete
 - programozói (API, rendszerhívások)
 - parancssori (illetve grafikus)
- Fájlrendszerek szervezési felülete
 - diszk szervezés

UNIX fájlrendszerek történeti áttekintése

- System V első fájlrendszer s5fs
 - 80-as évek, alap implementáció, egyszerű szervezés
- 4.2 BSD Fast File Systems FFS (a Linux ext2 alapja is)
 - megnövelt teljesítmény
 - új szolgáltatások
 - akkori diszk hardver felépítéshez optimalizált rendszer
- Virtuális fájlrendszerek vnode/vfs
 - moduláris, objektum-orientált
 - cserélhető szervezési modulok, akár hálózati is
- Elosztott fájlrendszerek
 - NFS: transzparens hálózati fájlrendszer RPC megvalósítással
- Modern fájlrendszerek
 - ...

A fájlrendszer felhasználói szemmel

- Operációs rendszer felhasználó
 - parancssori és grafikus felület
 - könyvtárszervezés, speciális könyvtárak
 - fájlok és könyvtárak kezelése, attribútumaik
 - fájlrendszerek menedzselése (rendszergazda)
- Programozó (alkalmazás fejlesztő)
 - programozói interfészek (rendszerhívások, rendszerkönyvtárak)
 - fájlleírók, nyitott fájl objektumok és kezelésük
 - zárolási módszerek: kötelező, ajánlott

Fájlrendszerek szervezése

- Csatlakoztatás
 - csatlakoztatási pont
 - elfedés
- Szervezés a háttértáron
 - blokkos tárolás
 - fájlok leírói (diszk inode)
 - szabad helyek kezelése
- Szervezés a memóriában
 - csatlakoztatás nyilvántartása
 - fájlok leírói (memória inode)
 - kapcsolat a nyitott fájl objektumokhoz

A System V fájlrendszer

- Az eredeti verzió, minden szabad hozzáférésű és kereskedelmi verzió támogatja
- A fájlokat logikai szinten és nem diszkszinten kezeli
- Logikai blokkok mérete: $512 \cdot 2^k$ bájt ($k = 0 \dots 5$)
- A diszken elhelyezett fájlrendszer részei
 - (boot blokk (rendszer induláshoz adatok – csak 1 állományrendszernek kell, a többinél üres))
 - superblokk (fájlrendszer metaadatok)
 - inode lista (fájl metaadatok)
 - Adatblokk (tárolt adatok)



Szuperblokk

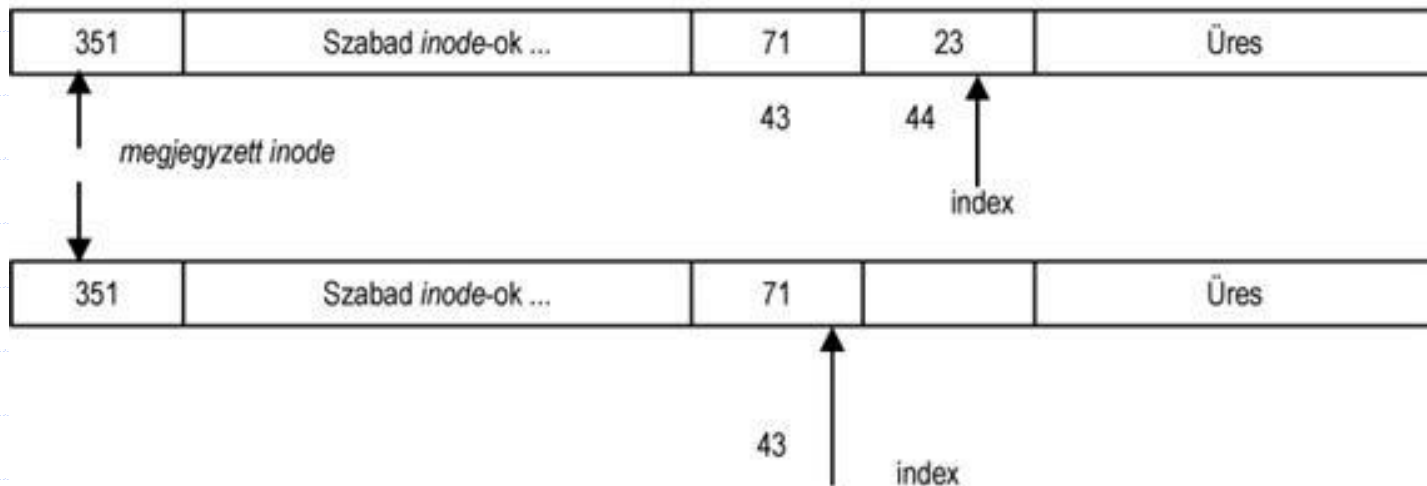
- A rendszer működéséhez szükséges metaadatok
- Szuperblokk
 - a fájlrendszer mérete
 - a szabad blokkok száma,
 - a szabad blokkok listája,
 - a következő szabad blokk indexe,
 - az inode lista mérete,
 - a szabad inode-ok száma,
 - inode lista (tömb),
 - következő szabad inode indexe,
 - lock mezők (inode, blokklista),
 - módosítás jelzőbit.

Az index node (inode)

- az állományrendszer a superblokk után egy lineáris listában tárolja az inode-okat
- Minden állományhoz 1 inode tartozik, ezek alapján lehet az állományokra hivatkozni (a rendszergazda állítja be a méretét)
- Hatékonyság miatt a superblokk tartalmaz egy fix méretű tömböt (szuperblokk inode lista), amelyet gyorsítótárként (cache) használ a szabad inode-okhoz
 - hitelesítési információk (UID, GID)
 - típus (értéke 0 ha az inode szabad)
 - hozzáférési jogosultságok
 - időbélyegek
 - méret
 - adatblokkok elhelyezkedése (címtábla)
 - 10-15 db direkt blokkcím
 - 1x, 2x és 3x indirekt blokkcímek

Számítási példa – a szb-ban tárolt inode lista kezelése

- Inode lista - az állományrendszer a szuperblokk után egy lineáris listában tárolja az inode-okat+ a szuperblokkban cache-li a szabad inode listát
- Ha a rendszernek szabad inode-ra van szüksége (például fájl létrehozása), keresés helyett kiveszi a tömbben tárolt utolsó elemet (index mutat rá)



Számítási példa – a szb-ban tárolt inode lista kezelése (folyt.)

- Ha kiürül a szuperblokk inode listája, a kernel elkezd vizsgálni az (igazi) inode listát, szabad inode-okat keres, és hátulról kezdve teljesen feltölti a szuperblokkon belüli inode listát.
- Az utolsó megtalált szabad inode sorszám kerül a tömb első helyére (megjegyzett inode, a következő keresés innen indul)
- ha egy inode felszabadul: van szabad hely a tömbben?
 - Igen: beteszi a sorszámot a tömb első szabad helyére, és eggyel növeli az indexet
 - Nem: megjegyzett inode kisebb, mint a felszabadítandó?
 - Igen: eldobja, (úgy is megtalálja később)
 - Nem: kicseréli a megjegyzett inode-dal (innen kezd majd keresni)

Számítási példa – szabad lemezblokkok listája

- Tudni kell, hogy szabad vagy nem?
- Van szabad blokkok listája, az utolsó egy blokkra mutat, amelyben szabad blokkok vannak tárolva. Pl. blokk méret 1k, 4 bájtos sorszárok $\rightarrow 1024/4 = 256$ szabad blokk tárolható egy blokkban

105							
-----	--	--	--	--	--	--	--

(szuperblokk)

214	206	203	217	...			
-----	-----	-----	-----	-----	--	--	--

(105-ös blokk)

105	934						
-----	-----	--	--	--	--	--	--

(felszabadul a 934-es blokk)

214	206	203	217	...			
-----	-----	-----	-----	-----	--	--	--

(szuperblokk, miután 2 db szabad blokkra volt szükség (934, 105))

	315	317	321	...			
--	-----	-----	-----	-----	--	--	--

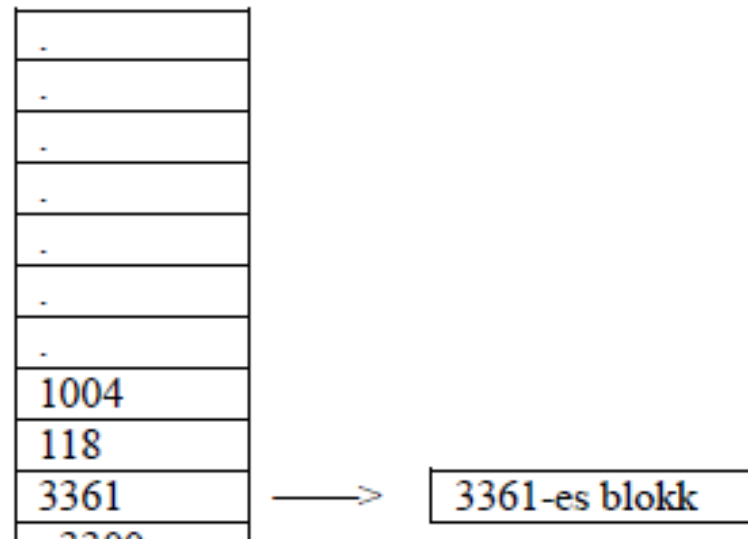
(214-es blokk)

Adatblokkok, a reguláris állományok szerkezete

- Az adatok elhelyezkedését az inode-ban található címtábla (13 mutató) írja le.
- Indexelt allokáció egy speciális formája, blokkonként allokál
- Rugalmas, jól kezelhető
- Pl. diszk blokkok: 1k, blokk cím: 4 bájt
- 10 mutató (direkt blokkok) diszk blokkra mutat ($10 \cdot 1k$)
- 1 mutató (egyszeres indirekt) egy olyan blokkra, amelyben diszk blokkok vannak (256k)
- 1 mutató (kétszeresen indirekt) egy olyan blokkra, amelyben olyan blokkok vannak, amelyekben diszk blokkok vannak (64M)
- 1 mutató (háromszorosan indirekt) egy olyan blokkra, amelyben olyan blokkok vannak, amelyben olyan blokkok vannak, amelyekben diszk blokkok vannak (16G)
- Elméletben gyakorlatban 4G az állományméret

Számítási példa1: a reguláris állományok szerkezete

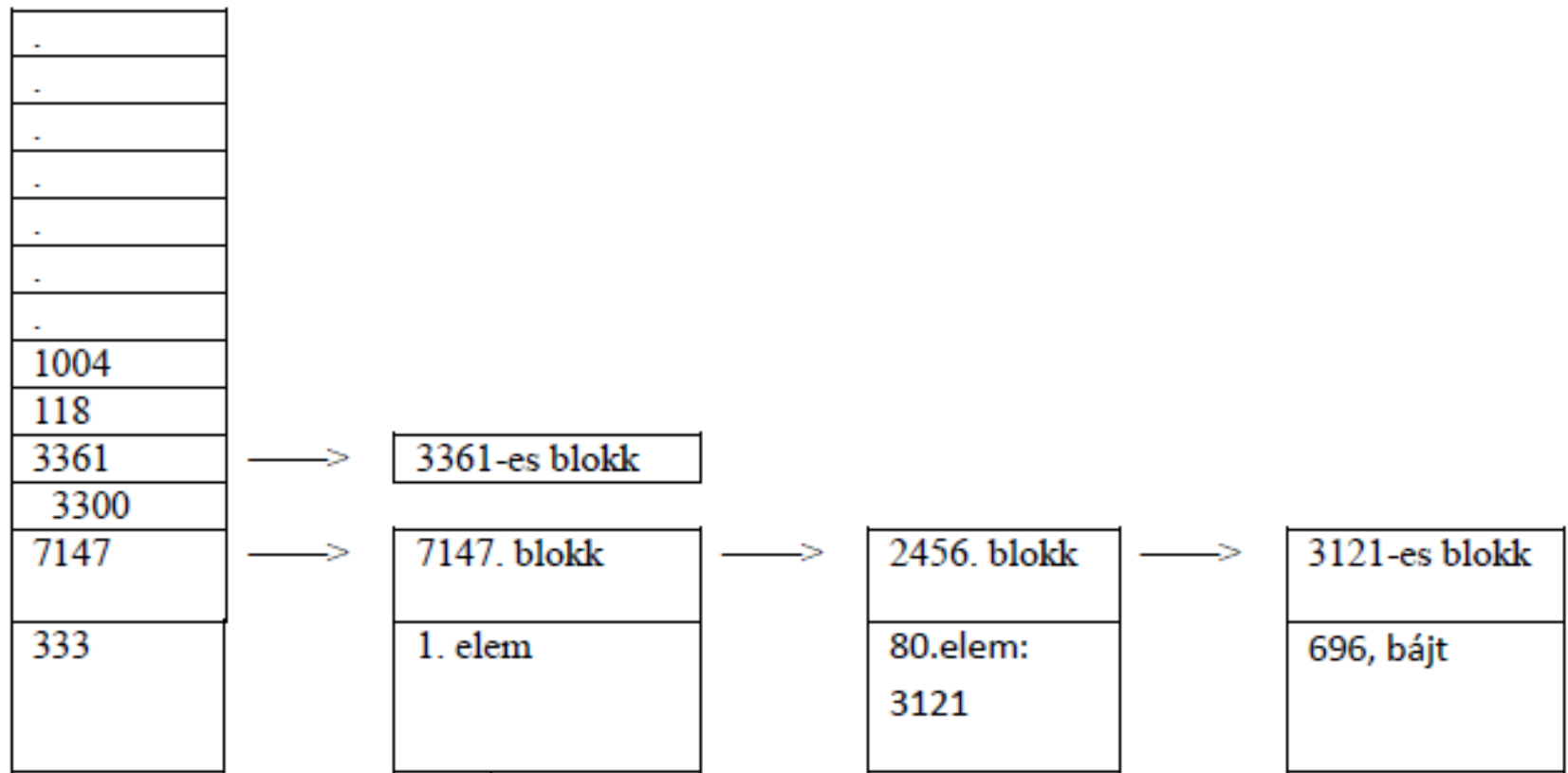
- Egy fájl 9500-adik byte-ját akarjuk olvasni, amihez a kernelnek meg kell találni az ehhez tartozó blokk címet.
- A 9500-adik byte a 9. direkt blokkban van ($9 \cdot 1024 = 9216$), ott pedig a 284-edik (a blokkokat 0-tól kezdődően sorszámozzuk). A példában az inode 9. direkt blokkra mutató pointer pl. a 3361. lemezblokkra mutat, vagyis a 3361. blokk 284. byte-ja a keresett adat.



Számítási példa2: a reguláris állományok szerkezete

- Tegyük fel, hogy a 355 000-es byte-ot akarjuk olvasni.
- Ehhez kétszeres indirekt blokkot kell használni, mert a 10 direkt és az egyszeres indirekt blokkal $10 \cdot 1024 + 256 \cdot 1024 = 272384$ byte-ot lehet elérni. Innen számítva a 82616. byte-ot keressük. Az a kétszeres indirekt blokk első (0-ás indexű) mutatója által kijelölt egyszeres indirekt blokkban van, méghozzá annak 80. elemében ($80 \cdot 1024 = 81920$). Az így kijelölt blokk 696. byte-ja a keresett adatelem. A fenti példa esetén a kétszeres indirekt blokk pointere a 7147. diszk blokkra mutat, annak az első pointere a 2456. blokkra, ami tartalmazza 80. elemként a keresett adatokat tartalmazó blokk címét, a 3121-et. Ezen a blokkon belül a 696. byte-ot kerestük.

Számítási példa2: a reguláris állományok szerkezete



Az állománykezelés logikai sémája és adatszerkezetei

- Név szerinti hivatkozás: csak megnyitáskor (logikai perifériaszámot ad vissza)
- Egy folyamat többször is megnyithat egy fájlt (új, önálló logikai periféria jön létre)
- Több folyamat megnyithatja és használhatja ugyanazt a fájlt párhuzamosan (szinkronizáció!)
- A gyermek folyamatok öröklik a szülők logikai perifériáit

Az állománykezelés logikai sémája és adatszerkezetei (folyt.)

- 3 szintű adatszerkezet
- Folyamatonkénti állományleíró tábla (1 /folyamat)
 - A tábla egy bejegyzése egy logikai perifériának felel meg.
 - A táblában minden, a folyamat által elérhető, megnyitott állományhoz legalább egy mutató tartozik. A mutató a globális állománytábla megfelelő elemére mutat.
- Globális állománytábla (1 bejegyzés/megnyitás)
 - az állományok osztott használatát teszi lehetővé
 - megnyitás módja, jogosultságok, fájlmutató, hogy hol tartunk az állományban az olvasással/írással, hivatkozásszámláló, amely megmondja, hogy hány folyamatonkénti állományleíró tábla bejegyzés mutat rá
- Inode tábla (in-core inode tábla) (1/megnyitott fájl)
 - hivatkozásszámláló, hogy az adott nevű fizikai állományt hány példányban használják, vagyis hány globális állománytábla bejegyzés mutat rá

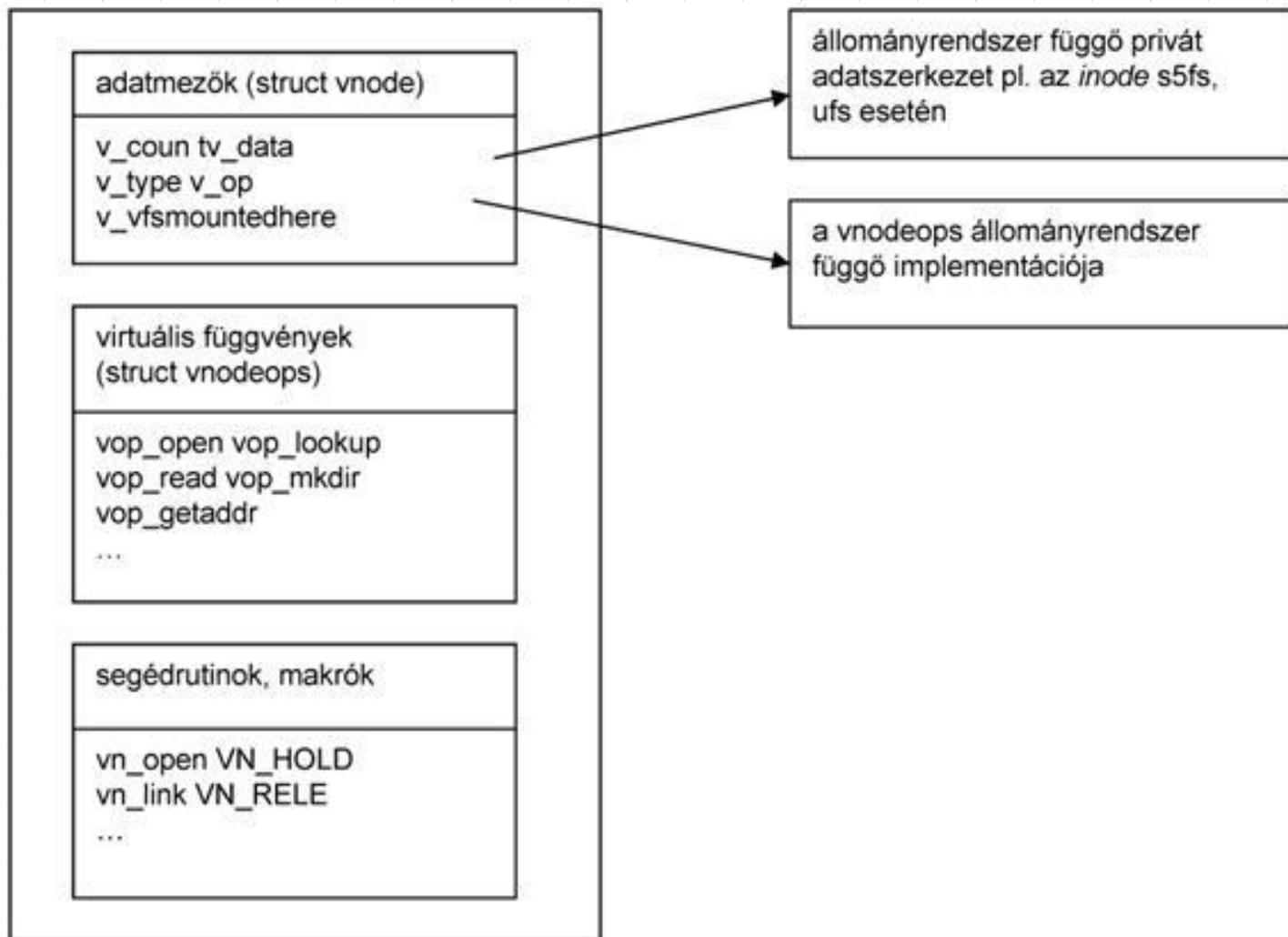
A Berkeley FFS fájlrendszer

- Az s5fs hiányosságait próbálja orvosolni
- Megtartja a s5fs minden funkcionalitását, a kernel adatszerkezetek nagy részét, és a rendszerhívások kezelési algoritmusát
- A legjelentősebb átalakítások:
 - lemez használatban, a lemezen lévő adatszerkezeteken és a szabad blokk allokáción
 - Bevezeti a cilindercsoportokat (valahány folytonos cylinder)
 - Biztonság növelése: a szuperblokkról minden egyes cylinder csoport tartalmaz egy másolatot
 - Belső fragmentáció ellen: a fájl utolsó blokkja lehet töredékblokk ($1/2$, $1/4$, $1/8$, de min. 512 bájt)
 - Egyéb bővítések: Hosszú fájl nevek, quota mechanizmus, szimbolikus link a hard link helyett (speciális állomány)
- CPU kihasználtság 4x

A Berkeley FFS fájlrendszer

- Korai rendszer: 1 rendszer – 1 fájl rendszer
 - Ehhez elég hatékony az inode absztrakció
- Modern UNIX rendszerek: 1 rendszer – többféle fájlrendszer
 - Új típusú leírás kell
 - Többféle fájlrendszer támogatása
 - Csatlakoztatás után egységes kép
 - Osztott fájl kezelés támogatása a hálózaton keresztül is
 - Moduláris bővítés támogatása
- Virtuális csomópont (vnode) – az inode helyett
 - Adatmező
 - Típusmező (v_type, az állomány típusa), hivatkozásszámláló (v_count), mount info (v_vfs_mountedhere)
 - Virtuális függvények – implementáció független
 - Segédrutinok
 - +2 függvény v_data, v_op
- Virtuális fájlrendszer (vfs)

A vnode szerkezete



Vfs (virtuális fájlrendszer)

- Root állomány – erre mountolódik a többi
- Minden típushoz van egy vfs struktúra, a kezelési infoval
- minden egyes olyan virtuális csomópontban (vnode), amely egyben egy állományrendszer gyökér csomópontja, a VROOT jelzőbit be van billentve.
- Fájlrendszerek
 - Univerzális célú helyi fájlrendszer: s5fs, FFS
 - Távoli állományrendszer
 - Speciális célú fájlrendszerek: temporális állományrendszer, processzor állományrendszer (226-227. oldal)
 - Modern állományrendszerek
 - HW adottságokhoz illeszkedni kell – „Periféria korlátozott rendszerek”
 - Naplózó rendszerek (journaling)

Swapping, teljes folyamatok háttértárra írása

- a háttértár szervezése (swap device)
 - blokkos eszköz
 - több összefüggő blokkcsoport foglalása -> szabad terület tördelődése
 - olvasás/írás sebessége
- folyamatok háttértárra írása
 - A tárolás tömörsége szerinti optimalizálás:
 - egész terület allokálása – törlés
 - Felszabadított terület éppen illeszkedik – bejegyzés összevonása

Swapping, teljes folyamatok háttértárra írása (folyt.)

- folyamatok háttértárról memóriába történő beolvasása

Mikor?

- a kernel fork rendszerhívást hajt végre – új folyamatot hoz létre, és nincs elég memória,
- a kernel (s)brk rendszerhívást hajt végre – vagyis memória tartományt bővít,
- a verem dinamikusan bővül,
- 4. a kernel átütemez (korábban háttértárra írt folyamatnak kell a hely)
- A háttértárra írás, illetve a háttértárról való beolvasás speciális szabályai
- A háttértárra kiírandó folyamat kiválasztásával kapcsolatos problémák

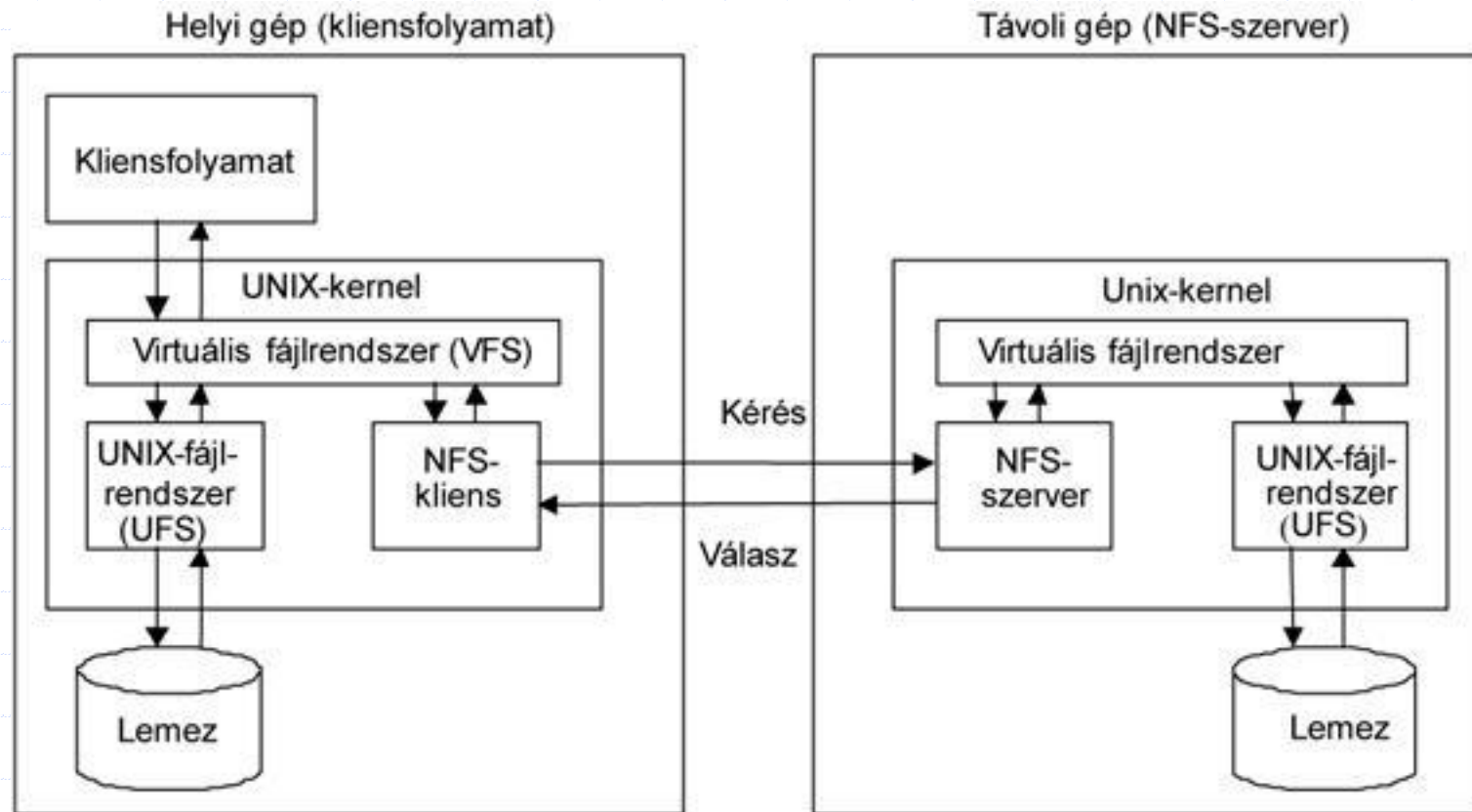
Virtuális memóriakezelés

- Elvárások
 - segítségével a fizikai memória méreténél nagyobb programokat lehessen futtatni,
 - csak részben memóriában lévő programok is futtathatók legyenek,
 - a multiprogramozásból adódóan támogatnia kell, hogy egyszerre több program is a memóriában lehessen,
 - támogassa áthelyezhető programok kezelését,
 - a memóriakezelés legyen gépfüggetlen,
 - vegye le a programozó válláról a memória allokációs és menedzselési terheket,
 - tegye lehetővé az osztott memória használatot.
- Virtuális memóriakezelés adatszerkezetei (352-360.old.)
- Copy-on-Write technika (361-362. o.)

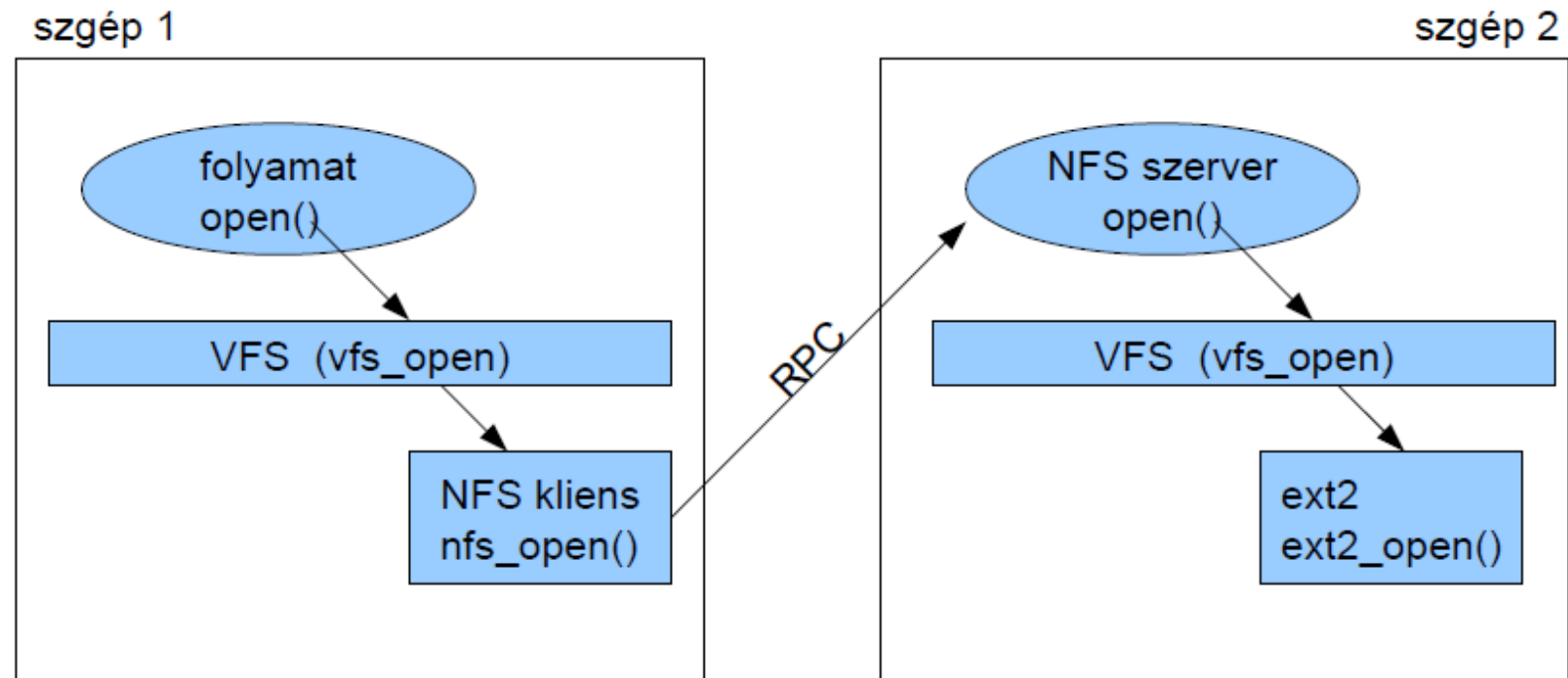
Hálózati és elosztott szolgáltatások a UNIX-ban

- 364-365., 218-220., 221-222., 225. 253. o.
- 1984-ben kiadott BSD UNIX-ba bekerült a TCP/IP protokoll támogatása (TCP/IP, OSI)
- SUN NFS (366-368, 372-373. o.)
 - Kliens-szerver modell
 - RPC technika
 - XDR adatábrázolási mód
 - Implementálás: Protokoll halmaz
- SW komponensek
 - NFS szerver
 - NFS kliens
 - Daemon folyamatok
 - NLM (Network Lock Manager)
 - NSM (Network Status Manager)

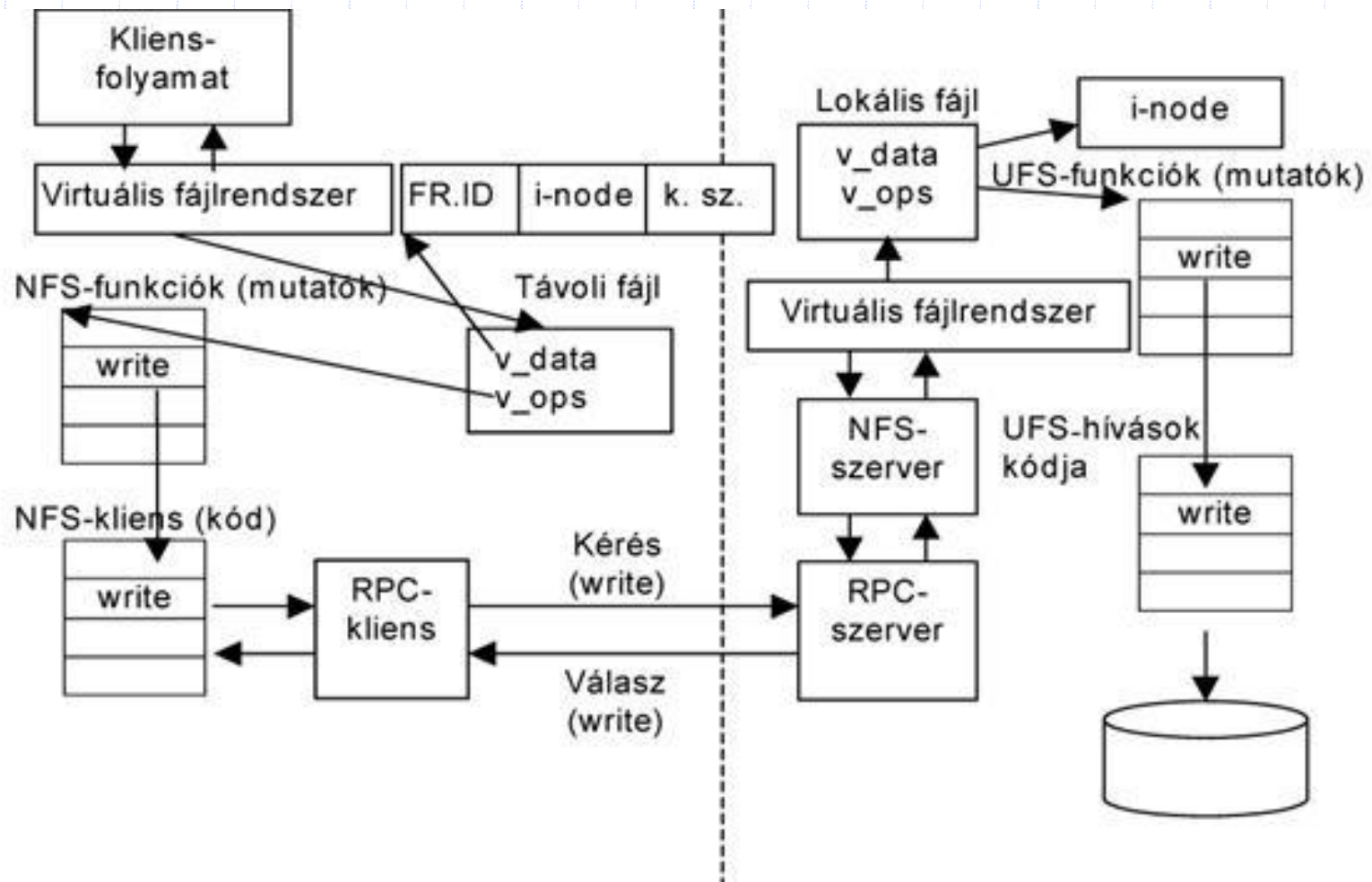
Az NFS működésének sémája



Alkalmazási példa: NFS egyszerűsített felépítés



Távoli fájl elérése NFS segítségével



Összefoglalás

- Alapok
- File rendszerek megvalósításai
 - System V
 - Berkeley FFS
 - Speciális fájlrendszerek
- Swapping
- Virtuális memóriakezelés
 - Copy-on-Write technika
- Hálózati és elosztott szolgáltatások a UNIX-ban
- virtuális fájlrendszer felépítés, alkalmazási példa: NFS